

docs.google.com – 全画面表示を終了するには、Esc を押します

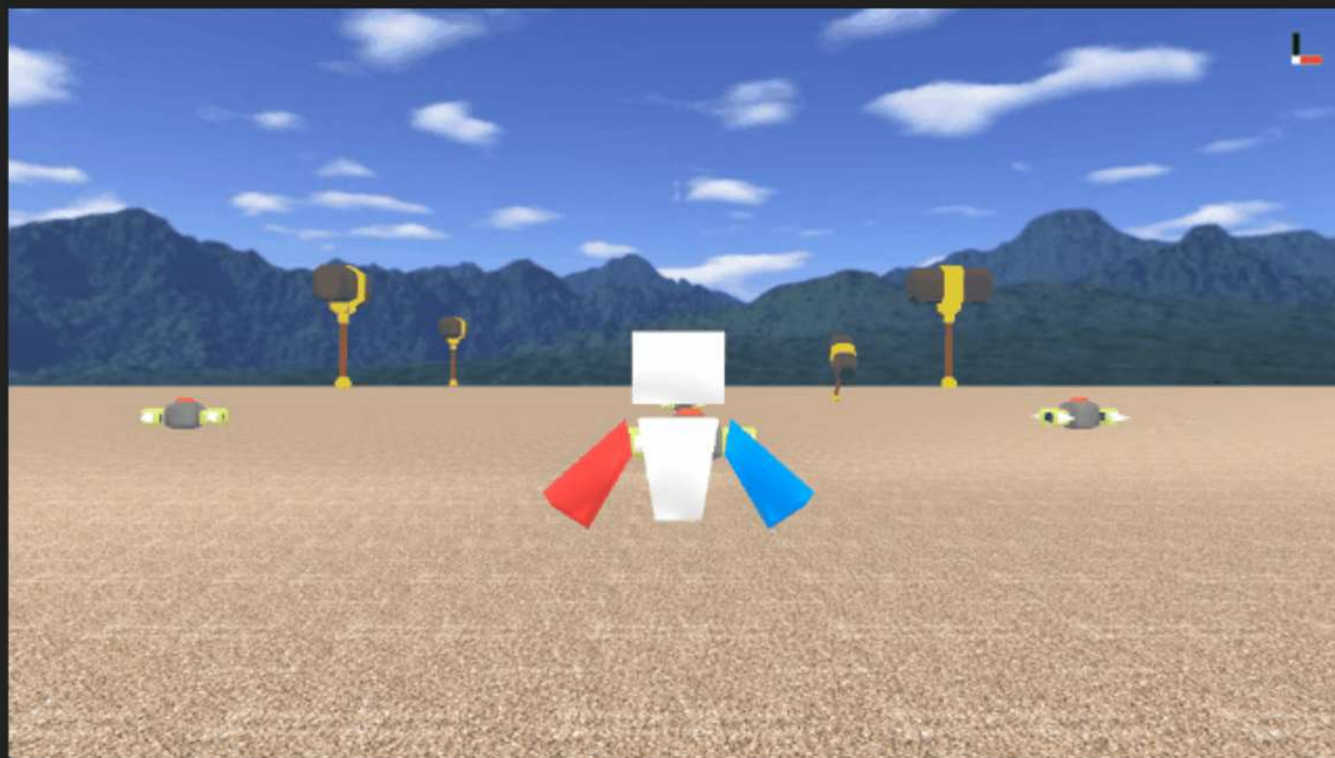
TL1

02_05.敵キャラSpawnPoint

日本工学院専門学校
デザインカレッジ

実行結果

日本工学院専門学校
デザインカレッジ



Blenderで背景物、自キャラに加えて敵も配置できるようになった。

ギミックの配置なども同様に行えばBlender上で異なるシンボルで配置することが可能だ。

レベルエディタを拡張して複雑なシーン配置を可能にしよう。

敵キャラの発生処理

日本工学院専門学校
デザインカレッジ

ゲームプレイシーンなど

```
// レベルデータから敵を生成、配置
for (auto& enemyData : levelData->enemies) {
    // 敵の生成
    敵キャラの生成

    // 敵の初期化
    敵キャラの初期化
    enemy->座標設定(enemyData.translation);
    enemy->回転角設定(enemyData.rotation);

    // 敵リストに追加
    enemies_.push_back(std::move(enemy));
}
```

読み込んだ配置データを元に、
敵を生成・配置していく。

なお、左のサンプルプログラムでは
fileName を使っていないが、
この文字列を使って生成する敵を変えたり
モデルを差し替えたりすることが可能なはずだ。

実際のゲーム制作でかなり役立つので
記憶に留めておこう。

オブジェクトの走査

日本工学院専門学校
デザインカレッジ

レベルデータローダー

```
// "objects"の全オブジェクトを走査
for (...省略...) {

    ...省略...

    // 自キャラ発生ポイント
    else if (type.compare("PlayerSpawn") == 0) {
        ...省略...
    }

    // 敵キャラ発生ポイント
    else if (type.compare("EnemySpawn") == 0) {
        enemiesに要素を1つ追加する
        平行移動の数値を書き込む
        回転角の数値をラジアンで書き込む
        file_nameがあれば書き込む
    }
}
```

Jsonファイルのデータを走査して記録する処理が既にあるが、そこに"**type**" 欄が "**PlayerSpawn**" の場合の処理を追加する。

といっても**PlayerSpawnData**に特別なデータはないので、背景オブジェクトについての処理とほとんど変わらない。

独自のデータを追加した場合は、ここで吸収することになる。

データ構造体

日本工学院専門学校
デザインカレッジ

レベルデータローダー

```
// 敵キャラの生成データ
struct EnemySpawnData {
    // ファイル名
    std::string fileName;
    // 平行移動
    KamataEngine::Vector3 translation;
    // 回転角
    KamataEngine::Vector3 rotation;
};
```

書き出したレベルデータから
実際にゲームに配置する処理を書いていく。

自キャラの配置とやることはほとんど変わらない。

ただしここでは敵の種類を区別するために
ファイル名もデータとして追加しておく。

レベルデータローダー

```
// レベルデータ
struct LevelData {
    ...省略...

    // 敵配列
    std::vector<EnemyData> enemies;
};
```

+

ゲーム側の対応

実行結果

日本工学院専門学校
デザインカレッジ

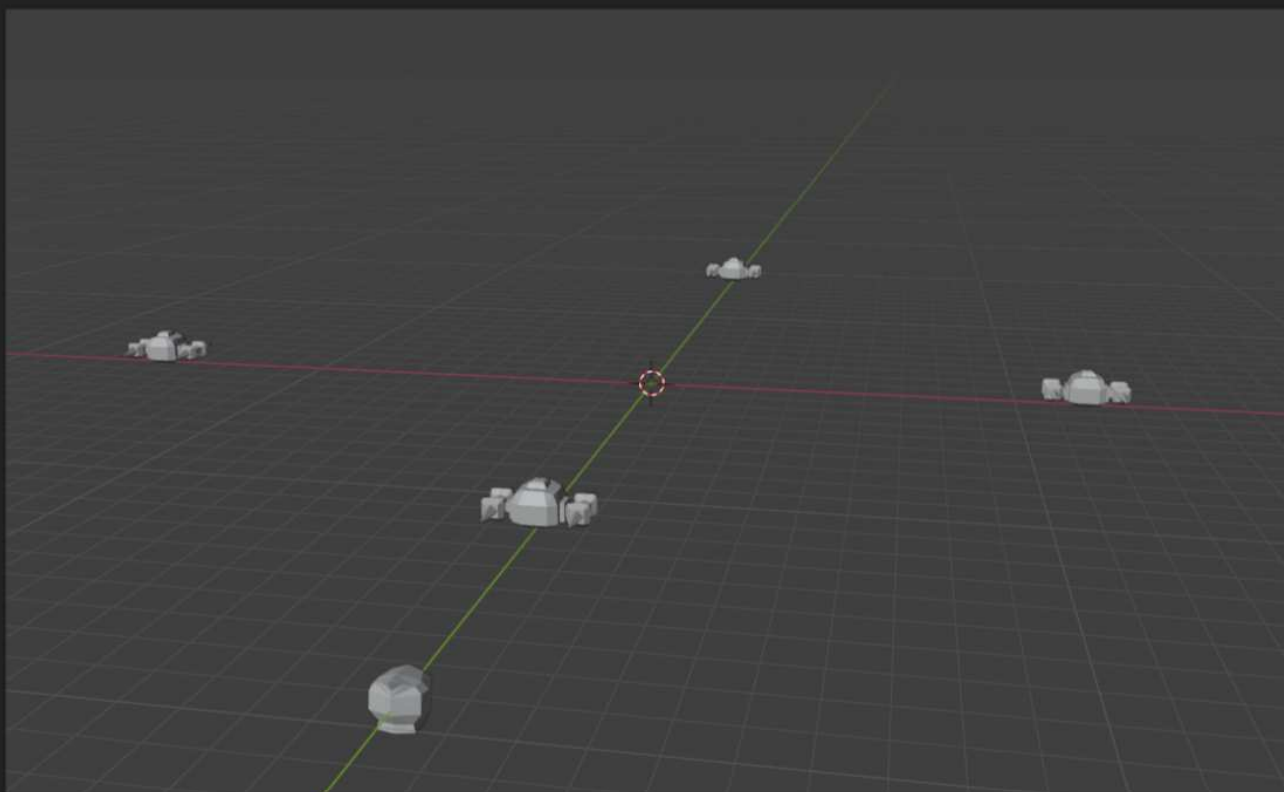
```
{  
  "type": "EnemySpawn",  
  "name": "EnemySpawn.003",  
  "transform": {  
    "translation": [  
      0.0,  
      -11.677591323852539,  
      0.0  
    ],  
    "rotation": [  
      0.0,  
      -0.0,  
      0.0  
    ],  
    "scaling": [  
      1.0,  
      1.0,  
      1.0  
    ]  
  }  
},
```

```
{  
  "type": "PlayerSpawn",  
  "name": "PlayerSpawn",  
  "transform": {  
    "translation": [  
      0.0,  
      -19.381359100341797,  
      0.0  
    ],  
    "rotation": [  
      0.0,  
      -0.0,  
      0.0  
    ],  
    "scaling": [  
      1.0,  
      1.0,  
      1.0  
    ]  
  }  
}
```

"PlayerSpawn" と "EnemySpawn" が
複数配置できて、それがJsonファイルに反映さ
れていれば今のところ成功といってよい。

実行結果

日本工学院専門学校
デザインカレッジ



さて、ここまでできたら
実際に自キャラと敵キャラを
Blender上で配置して、
配置データのエクスポートまで
してみよう。

エクスポートは改造しない

日本工学院専門学校
デザインカレッジ

```
def parse_scene_recursive_json(self, data_parent, object, level):
```

export_scene.py

```
    #オブジェクト種類
```

```
    if "type" in object: # カスタムプロパティで指定された場合
        json_object["type"] = object["type"]
```

```
    else:
```

```
        json_object["type"] = object.type
```

前回と同じ順序でいくなら、次はJsonファイルへのエクスポート機能の改造だ。

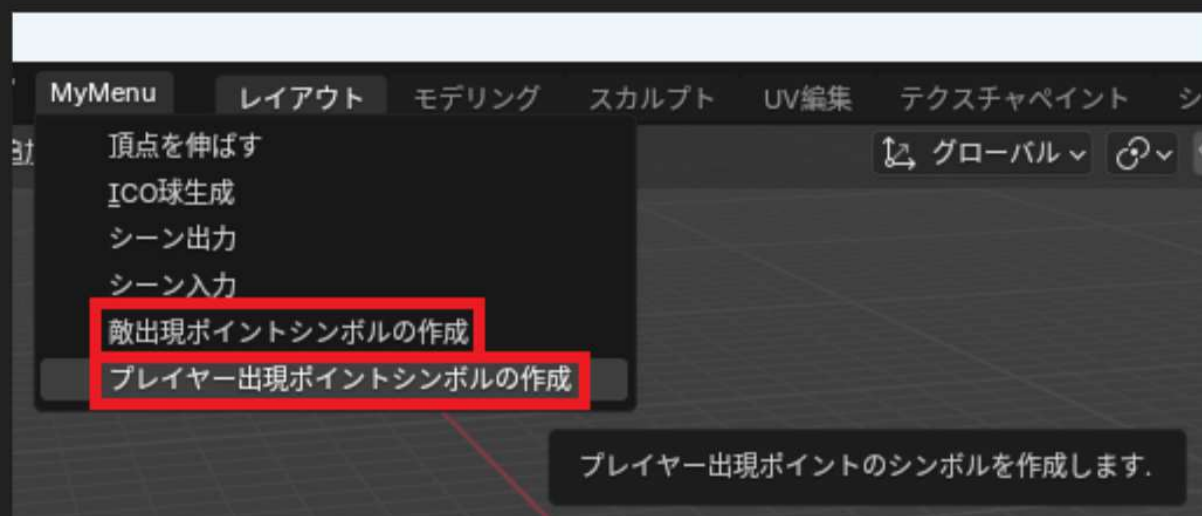
しかしここであらためて現在のプログラムを見返してみよう。
今回のテキストでの改造は **"type"** カスタムプロパティの値が
"PlayerSpawn" と **"EnemySpawn"** の2種類に増えるだけなので、
エクスポート処理はこのまま使える。

したがって改造の必要はない。

シーンデータのエクスポート

実行結果

日本工学院専門学校
デザインカレッジ



シンボルの作成コマンドが、メニュー上でも自キャラと敵キャラ専用のコマンド（オペレータ）に分かれた。

ツールとしてはこの方が便利だろう。

メニューに登録する

日本工学院専門学校
デザインカレッジ

my_menu.py

```
# トップバーの拡張メニュー
class 自作メニュー(bpy.types.Menu):

    # サブメニューの描画
    def draw(self, context):
        # トップバーのサブメニューにシンボル作成オペレータを追加
        自分で考えよう
```

変更

Blenderのメニュー登録を変更する。

前回既に「出現ポイントシンボル作成オペレータ」を登録したが、
自キャラと敵キャラ用にオペレータを分けたので、
「出現ポイントシンボル作成オペレータ」の登録を削除して、かわりに
自キャラと敵キャラシンボル用のオペレータをそれぞれ登録する。

メニューに登録

モジュールのimportとクラスリスト登録

```
__init__.py
# モジュールのインポート
...
+ 自分で考えよう

# Blenderに登録するクラスリスト
classes = (
+     自分で考えよう
)
```

オペレータが増えたので、
Blender側から実行できるように
__init__.py への登録を忘れずにしておこう。

すでにシンボルの読み込みなどのオペレータは
登録済みなので、今回増やした分を追加で
登録する。

敵専用オペレータ

日本工学院専門学校
デザインカレッジ

```
class 敵専用出現ポイントシンボル作成オペレータ(bpy.types.Operator):  
  
    bl_idname = "myaddon.myaddon_ot_spawn_create_enemy_symbol"  
    bl_label = "敵出現ポイントシンボルの作成"  
    bl_description = "敵出現ポイントのシンボルを作成します"  
  
    def execute(self, context):  
  
        bpy.ops.myaddon.myaddon_ot_spawn_create_symbol('EXEC_DEFAULT', type = "Enemy")  
  
        return {'FINISHED'}
```

spawn.py

`type`の指定のみ敵用に変更して、敵専用のシンボル作成オペレータも新規作成する。

各種テキスト部分もテーブルから引っ張るようにしてもよいだろう。

自キャラ専用オペレータ

日本工学院専門学校
デザインカレッジ

```
class 自キャラ専用出現ポイントシンボル作成オペレータ(bpy.types.Operator):
```

spawn.py

```
    bl_idname = "myaddon.myaddon_ot_spawn_create_player_symbol"
```

```
    bl_label = "プレイヤー出現ポイントシンボルの作成"
```

```
    bl_description = "プレイヤー出現ポイントのシンボルを作成します"
```

```
    def execute(self, context):
```

```
        bpy.ops.myaddon.myaddon_ot_spawn_create_symbol('EXEC_DEFAULT', type = "Player")
```

```
        return {'FINISHED'}
```

シンボル作成オペレータにオプションを指定できるようにしたが、オペレータをBlenderのメニューに登録して呼び出すことを考えると **type** を限定するオペレータはそれはそれで欲しい。

ということで、上記のように自キャラ専用のシンボル作成オペレータを新規に作る。

自キャラ専用オペレータ

日本工学院専門学校
デザインカレッジ

```
class 自キャラ専用出現ポイントシンボル作成オペレータ(bpy.types.Operator):
```

spawn.py

```
    bl_idname = "myaddon.myaddon_ot_spawn_create_player_symbol"
```

```
    bl_label = "プレイヤー出現ポイントシンボルの作成"
```

```
    bl_description = "プレイヤー出現ポイントのシンボルを作成します"
```

```
    def execute(self, context):
```

```
        bpy.ops.myaddon.myaddon_ot_spawn_create_symbol('EXEC_DEFAULT', type = "Player")
```

```
        return {'FINISHED'}
```

シンボル作成オペレータにオプションを指定できるようにしたが、オペレータをBlenderのメニューに登録して呼び出すことを考えると **type** を限定するオペレータはそれはそれで欲しい。

ということで、上記のように自キャラ専用のシンボル作成オペレータを新規に作る。

プロパティの利用

日本工学院専門学校
デザインカレッジ

spawn.py

```
#オペレータ 出現ポイントのシンボルを作成・配置する
class 出現ポイントシンボル作成オペレータ(bpy.types.Operator):
    ...省略...

    # プロパティ (引数として渡せる)
    type: bpy.props.StringProperty(name="Type", default="Player")

    def execute(self, context):
        ...省略...

        # コピー元オブジェクトを検索
        spawn_object = bpy.data.objects.get(SpawnNames.names[self.type][SpawnNames.PROTOTYPE])
        if spawn_object is None:
            bpy.ops.myaddon.myaddon_ot_spawn_import_symbol('EXEC_DEFAULT')
            spawn_object = bpy.data.objects.get(SpawnNames.names[self.type][SpawnNames.PROTOTYPE])
```

変更

変更

プロパティはこのようなを `self.` をつけてメンバ変数として参照できる。この他にもオブジェクト名に関する処理があるので、データテーブルから値を引っ張ってくるように改造すること。

シンボル生成オペレータの改造

日本工学院専門学校
デザインカレッジ

変更

```
#オペレータ 出現ポイントのシンボルを作成・配置する
class 出現ポイントシンボル作成オペレータ(bpy.types.Operator):
    bl_idname = 適切に設定する
    bl_label = "出現ポイントシンボルの作成"
    bl_description = "出現ポイントのシンボルを作成します"
    bl_options = {'REGISTER', 'UNDO'}

    # プロパティ (引数として渡せる)
    type: bpy.props.StringProperty(name="Type", default="Player")

    def execute(self, context):
        ...省略...
```

spawn.py

シンボル生成オペレータの方も改造していく。

bpyにはオペレータの実行にあたってオプションを指定するためのプロパティという機能がある。

オプションの値にはデフォルト値が設定できるので、実行時にはオプションを指定してもしなくてもよい。

プロパティはメンバ関数内でメンバ変数として値を参照できる。

シンボルの配置（作成）

新たなexecute処理

日本工学院専門学校
デザインカレッジ

spawn.py

```
#オペレータ 出現ポイントのシンボルを読み込む
class スポーンポイントシンボル読み込みオペレータ(省略):
    ...省略...

    def load_obj(self, type):
        ...省略...

    def execute(self, context):
        #Enemyオブジェクト読み込み
        self.load_obj("Enemy")
        #Playerオブジェクト読み込み
        self.load_obj("Player")

        return {'FINISHED'}
```

load_obj() に名前を変更したため、オペレータなのに execute() 関数がない状態になってしまった。

ということで、あらたな execute() 関数を用意する。その中で load_obj() を必要な回数呼び出すという寸法だ。

読み込み関数

日本工学院専門学校
デザインカレッジ

続き

```
def load_obj(self, type):
```

自分で考えよう

```
return {'FINISHED'}
```

読み込み関数内に、他にも何か所かオブジェクト名やファイル名に関する記述があるので、
そこも同様にデータテーブルから値を引っ張ってくるように改造する。

読み込み関数

日本工学院専門学校
デザインカレッジ

変更

spawn.py

```
#オペレータ 出現ポイントのシンボルを読み込む
class スポーンポイントシンボル読み込みオペレータ(省略):
    ...省略...

    def load_obj(self, type):

        # 重複ロード防止
        spawn_object = bpy.data.objects.get(SpawnNames.names[type][SpawnNames.PROTOTYPE])
        if spawn_object is not None:
            return {'CANCELLED'}
        ...省略...
```

変更

`execute()` 関数内にシンボルのモデル読み込み処理を直接書いていたが、
`load_obj()` 関数と名をあらため、1モデル分の読み込み処理ということにする。

これによって、`load_obj()` 関数を複数回呼び出すことで、自キャラや敵など複数種類のモデルの読み込みを可能にする狙いだ。

シンボルの読み込み

データテーブル

日本工学院専門学校
デザインカレッジ

```
class SpawnNames():  
  
    # インデックス  
    PROTOTYPE = 0    # プロトタイプのオブジェクト名  
    INSTANCE = 1     # 量産時のオブジェクト名  
    FILENAME = 2     # リソースファイル名  
  
    names = {}  
    # names["キー"] = (プロトタイプのオブジェクト名、量産時のオブジェクト名、リソースファイル名)  
    names["Enemy"] = ("PrototypeEnemySpawn", "EnemySpawn", "needle/needle.obj")  
    names["Player"] = ("PrototypePlayerSpawn", "PlayerSpawn", "player/player.obj")
```

spawn.py

バリエーションを出すときの基本原則は、なるべく if文や switch文 による分岐をさけ、処理は共通として種類ごとに異なる部分をデータとして分離し、データテーブル化することである。

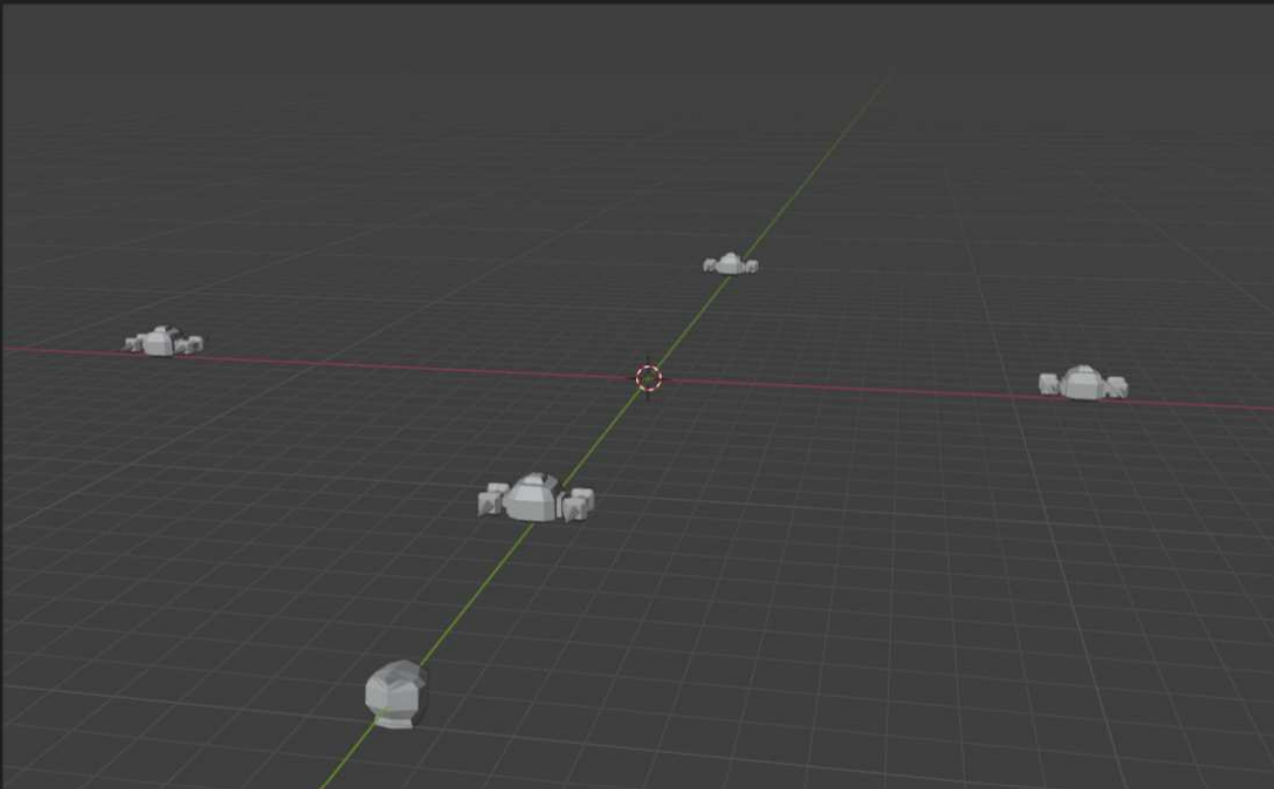
今回は実装にあたり、C++ の namespace のような意味合いでデータをまとめるためのクラスを作る。

C++なら1種類分のデータを構造体にまとめるところだが、pythonなのでいきなり代入する。

データテーブル

今回の目標

日本工学院専門学校
デザインカレッジ



今回はフィールド上での
敵キャラの出現位置を
Blender上でSpawnPoint
として配置できるようにする。

なお、プログラミングの原則と
して、単純なコピペによる
実装でなく、処理の共通化を図
る。

TL1

02_05.敵キャラSpawnPoint